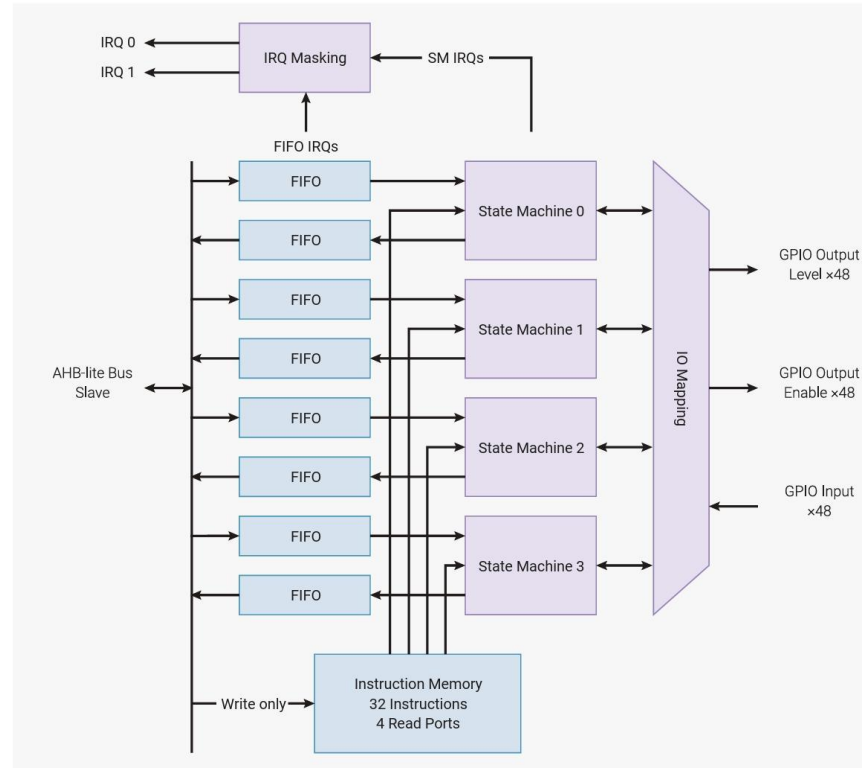


Instrucciones PIO

Figure 43. PIO block-level diagram. There are three PIO blocks, each containing four state machines. The four state machines simultaneously execute programs from shared instruction memory. FIFO data queues buffer data transferred between PIO and the system. GPIO mapping logic allows each state machine to observe and manipulate up to 32 GPIOs.



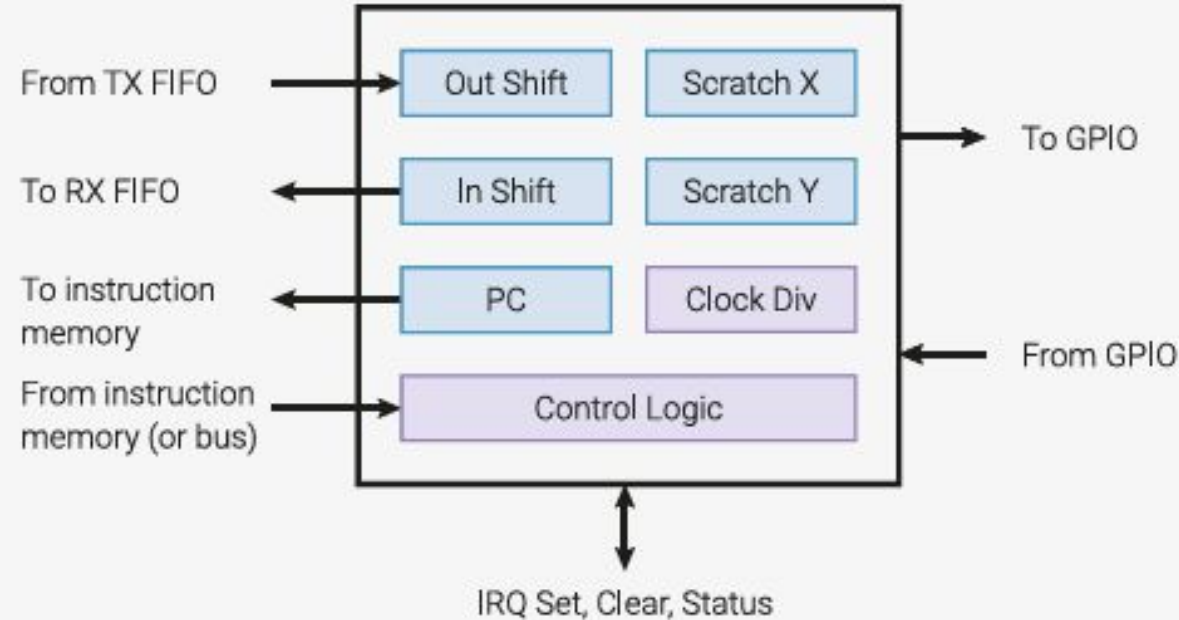
RP2040

The clock divider is computed as $\text{system clock frequency} / \text{freq}$, so there can be slight rounding errors.

The minimum possible clock divider is one 65536th of the system clock: so at the default system clock frequency of 125MHz, the minimum value of *freq* is 1908. To run state machines at slower frequencies, you'll need to reduce the system clock speed with

`machine.freq()`.

RP2040 125MHz
RP2350 150MHz



PIO instructions are 16 bits long, and use the following encoding:

Table 977. PIO
instruction encoding

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
JMP	0	0	0	Delay/side-set					Condition			Address				
WAIT	0	0	1	Delay/side-set					Pol	Source		Index				
IN	0	1	0	Delay/side-set					Source			Bit count				
OUT	0	1	1	Delay/side-set					Destination			Bit count				
PUSH	1	0	0	Delay/side-set					0	IfF	Blk	0	0	0	0	0
MOV	1	0	0	Delay/side-set					0	0	0	1	IdxI	0	Index	
PULL	1	0	0	Delay/side-set					1	IfE	Blk	0	0	0	0	0
MOV	1	0	0	Delay/side-set					1	0	0	1	IdxI	0	Index	
MOV	1	0	1	Delay/side-set					Destination			Op		Source		
IRQ	1	1	0	Delay/side-set					0	Clr	Wait	IdxMode		Index		
SET	1	1	1	Delay/side-set					Destination			Data				

All PIO instructions execute in one clock cycle.

$$\text{bits(Delay)} + \text{bits(side-set)} = 5$$

$$\text{sideset_count} = \text{bits(side-set)}$$

```
from machine import Pin
import rp2
```

```
# Create and start a StateMachine
# with blink_1hz, outputting on Pin(25)
sm = rp2.StateMachine(
    0,
    blink_1hz,
    freq=2300,
    set_base=Pin(25)
)
sm.active(1)
```

```
@rp2.asm_pio(set_init=rp2.PIO.OUT_LOW)

def blink_1hz():
    # Cycles: 1 + 7 + 32 * (30 + 1) = 1000
    set(pins, 1)
    set(x, 31) [6]
    label("delay_high")
    nop() [29]
    jmp(x_dec, "delay_high")

    # Cycles: 1 + 7 + 32 * (30 + 1) = 1000
    set(pins, 0)
    set(x, 31) [6]
    label("delay_low")
    nop() [29]
    jmp(x_dec, "delay_low")
```

`nop()`

This is a pseudoinstruction that assembles to `mov(y, y)` and has no effect.

`.side(value)`

This is a modifier which can be applied to any instruction, and is used to control side-set pin values.

- value: the value (bits) to output on the side-set pins

`nop().side(1)`

`.delay(value)`

`[value]`

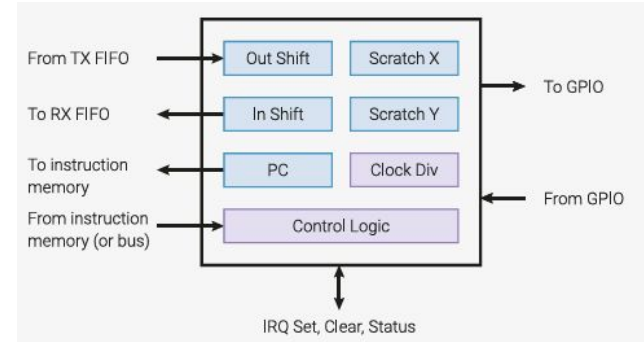
This is a modifier which can be applied to any instruction, and specifies how many cycles to delay for after the instruction executes.

- value: cycles to delay, 0-31 (maximum value reduced if side-set pins are used)

`nop() [31]`

`nop().delay(31)`

$\text{bits}(\text{Delay}) + \text{bits}(\text{side-set}) = 5$
 $\text{sideset_count} = \text{bits}(\text{side-set})$



`wrap_target()`

Specify the location where execution continues after program wrapping. By default this is the start of the PIO routine.

`wrap()`

Specify the location where the program finishes and wraps around. If this directive is not used then it is added automatically at the end of the PIO routine. Wrapping does not cost any execution cycles.

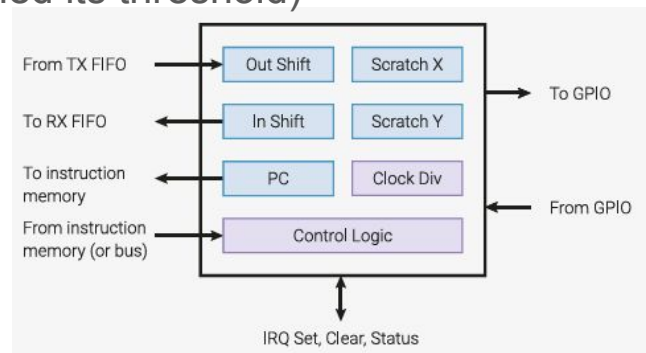
`jmp(label), jmp(cond, label)`

`cond`: the condition to check, one of:

- `not_x, not_y`: true if register is zero
- `x_dec, y_dec`: true if register is non-zero, and do post decrement
- `x_not_y`: true if X is not equal to Y
- `pin`: true if the input pin is set
- `not_osre`: true if OSR is not empty (hasn't reached its threshold)

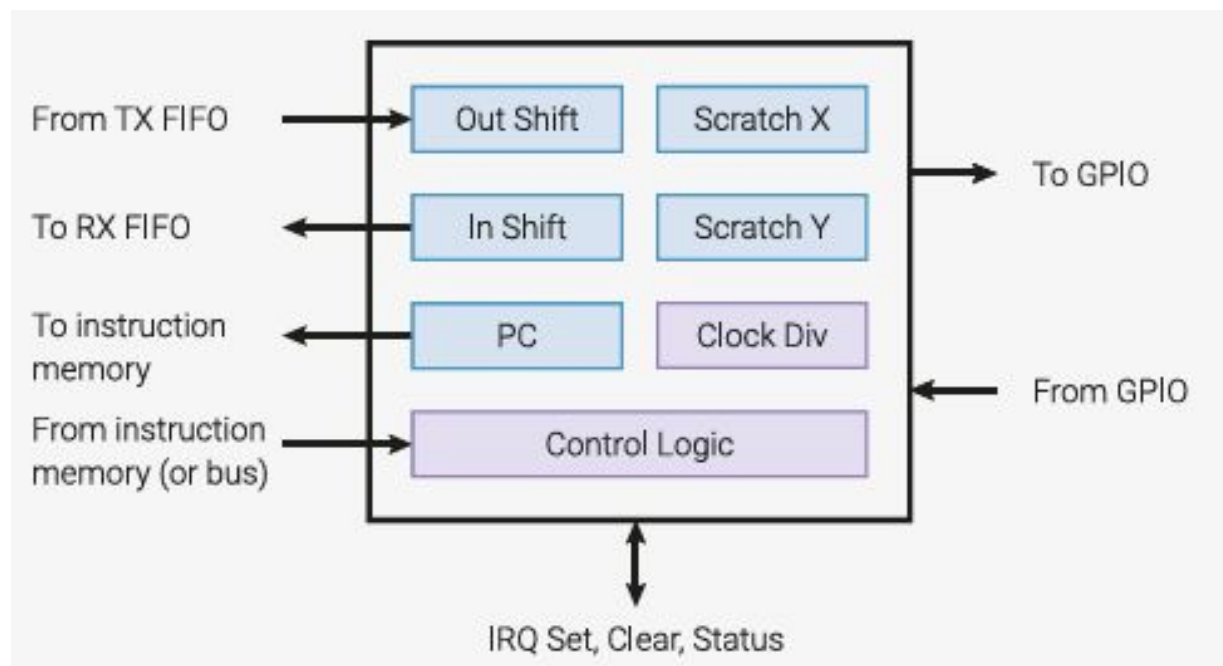
`label`: label to jump to if condition is true

`pin` is defined by `jmp_pin (rp2.StateMachine)`



`label(label)`

Define a label called `label` at the current location. `label` can be a string or integer.



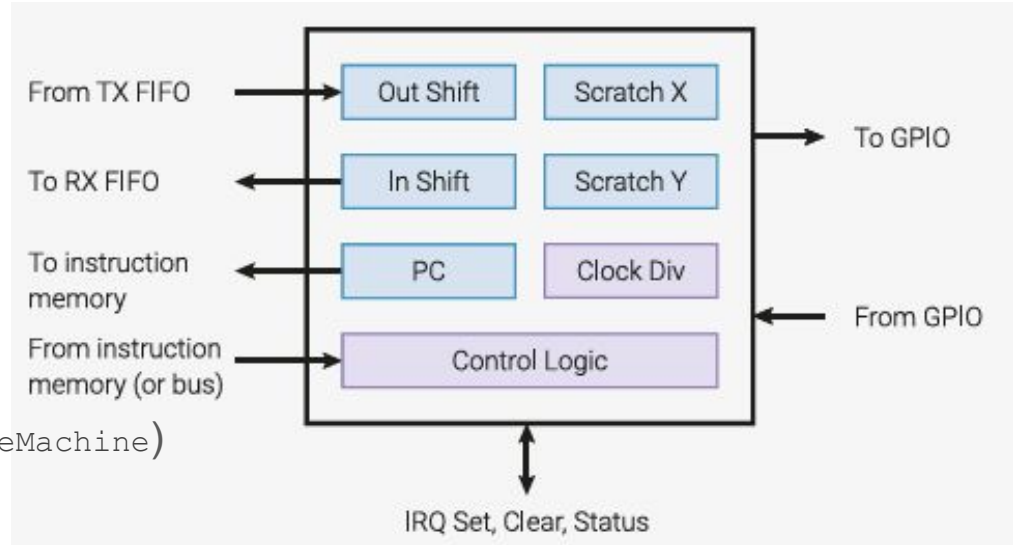
`set(dest, data)`

Set dest with the value data.

dest: pins, x, y, pindirs

data: value (0-31)

pins are defined by `set_base (rp2.StateMachine)`



`mov(dest, src)`

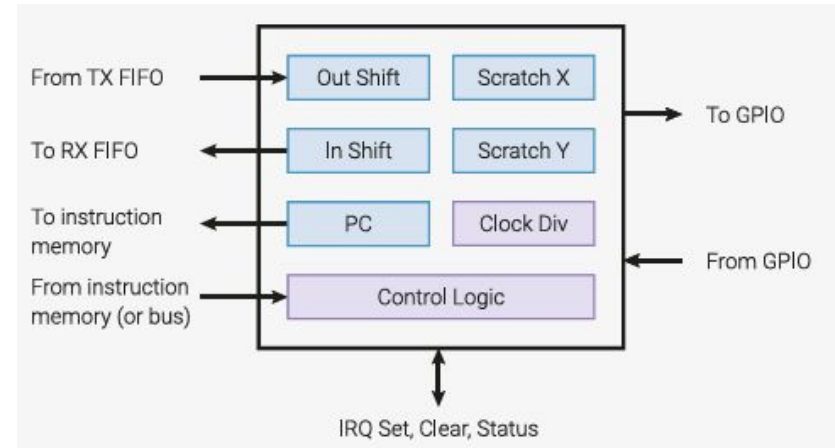
Move into dest the value from src.

- dest: one of: pins, x, y, exec, pc, isr, osr
- src: one of: pins, x, y, null, status, isr, osr;

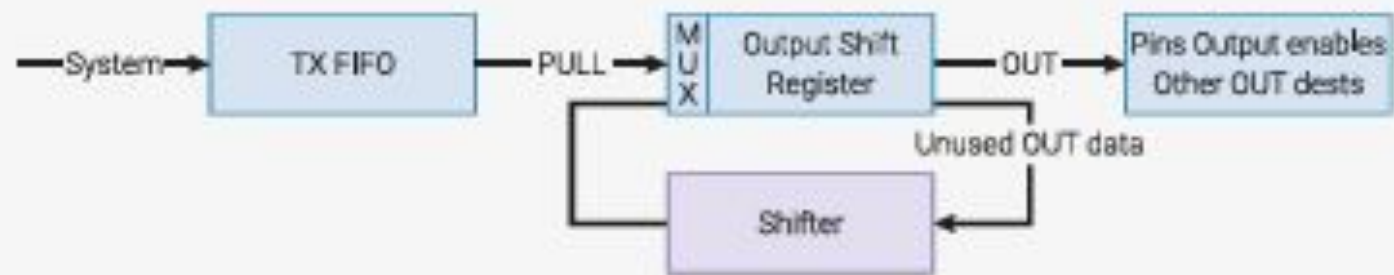
this argument can be optionally modified by wrapping it in `invert()` or `reverse()` (but not both together)

`mov(x, invert(0))`

`dest_pins` are defined by `out_base` (`rp2.StateMachine`)
`src_pins` do not need definition ()







`in_(src, bit_count)`

Shift data in from src to ISR.

src: one of: pins, x, y, null, isr, osr

bit_count: number of bits to shift in (1-32)

out(dest, bit_count)

Shift data out from OSR to dest.

dest: one of: pins, x, y, pindirs, pc, isr, exec

bit_count: number of bits to shift out (1-32)

push(...)

Push ISR to the RX FIFO, then clear ISR to zero. This instruction takes the following forms:

push(), push(block), push(noblock)

push(iffull), push(iffull, block), push(iffull, noblock)

If block is used then the instruction stalls if the RX FIFO is full. The default is to block. If iffull is used then it only pushes if the input shift count has reached its threshold.

`pull(...)`

Pull from the TX FIFO into OSR. This instruction takes the following forms:

`pull()`, `pull(block)`, `pull(noblock)`

`pull(ifempty)`, `pull(ifempty, block)`, `pull(ifempty, noblock)`

If `block` is used then the instruction stalls if the TX FIFO is empty. The default is to `block`. If `ifempty` is used then it only pulls if the output shift count has reached its threshold.

irq(...)

Set or clear an IRQ flag. This instruction takes two forms:

irq(index)

index: 0-7, or rel(0) to rel(7)

irq(mode, index)

mode: one of: block, clear

index: 0-7, or rel(0) to rel(7)

If block is used then the instruction stalls until the flag is cleared by another entity. If clear is used then the flag is cleared instead of being set. Relative IRQ indices add the state machine ID to the IRQ index with modulo-4 addition. IRQs 0-3 are visible from to the processor, 4-7 are internal to the state machines.

`wait(polarity, src, index)`

Block, waiting for high/low on a pin or IRQ line.

polarity: 0 or 1, whether to wait for a low or high value

src: one of: gpio (absolute pin), pin (pin relative to StateMachine's `in_base` argument), irq

index: 0-31, the index for src

`word(instr, label=None)`

Insert an arbitrary 16-bit word in the assembled output.

`instr`: the 16-bit value

`label`: if given, look up the label and logical-or the label's value with `instr`

```
import time
from machine import Pin
import rp2
```

```
sm = rp2.StateMachine(0, blink_1hz, freq=2000,
set base=Pin(25))
```

```
sm.irq(lambda p: print(time.ticks_ms()))
```

```
# Start the StateMachine.
sm.active(1)
```

```
@rp2.asm_pio(set_init=rp2.PIO.OUT_LOW)
def blink_1hz():
    # Cycles: 1 + 1 + 6 + 32 * (30 + 1) = 1000
    irq(rel(0))
    set(pins, 1)
    set(x, 31)
    label("delay_high")
    nop()
    jmp(x_dec, "delay_high")

    # Cycles: 1 + 1 + 6 + 32 * (30 + 1) = 1000
    nop()
    set(pins, 0)
    set(x, 31)
    label("delay_low")
    nop()
    jmp(x_dec, "delay_low")
```